

Учебный симулятор диспетчера ДДС системы 112

Сопроводительная документация. Задача № 9, ЛЦТ-2026. Команда «Артифишл Интележенс». Редакция от 29.09.2026.

1. Назначение, функциональная и компонентная архитектура

1.1 Назначение

Учебный симулятор рабочего места диспетчера дежурно-диспетчерской службы (ДДС) системы 112 города Москвы (задача № 9, ЛЦТ-2026). Обучаемый в роли диспетчера ДДС получает готовую карточку происшествия из системы 112, принимает решение («Принята / Не принята»), выбирает бригаду, передаёт сведения через учебный IP-телефон, отмечает этапы реагирования и завершает карточку. Система оценивает действия по детерминированным правилам и локальной ИИ-модели, показывает преподавателю разбор и аналитику. Работает полностью локально, без интернета и без GPU.

1.2 Роли и функции

Роль	Основные функции	Где в интерфейсе
Обучаемый	АРМ ДДС: строка состояния с новыми карточками в реальном времени, карточка «формата 112», статусы с комментариями, выбор бригады, софтфон (набор 3-значного номера, реплики адресата, запись доклада), справочные материалы, свои результаты	/app/arm
Преподаватель	Занятия: состав и рабочие места, адресная выдача заданий пачкой, старт/стоп; live-доска и replay; вердикты и переопределения («согласен/не согласен»); отчёт и аналитика; студия сценариев (конструктор, предпросмотр, генерация и утверждение пакетов); учебные материалы	/app/teacher
Администратор	Учётные записи и роли, блокировка, нормативы и веса оценки, службы, состояние системы	/app/admin

Процесс занятия: преподаватель создаёт занятие (снимок настроек и методики времени) → выдаёт каждому рабочему месту свою очередь сценариев с задержками от старта → запускает занятие → worker в нужный момент создаёт карточку и оповещает АРМ → обучаемый работает с карточкой → терминальный статус создаёт мгновенную оценку по правилам и фоновую задачу ИИ-оценки → преподаватель видит разбор, переопределяет вердикт, выгружает отчёт.

Жизненный цикл карточки (универсальный для служб, памятка ДДС стр. 21–25): received → accepted | rejected; rejected → accepted (исправление) или перенаправление; accepted → responding → arrived → working → completed | refused, этапы только вперёд, пропуски допустимы (обязательные задаёт эталон сценария). Отказ без причины не принимается. Завершающий статус закрывает карточку для правок.

1.3 Компонентная архитектура

Компонент	Технология	Ответственность
Веб-клиент	React 18 + TypeScript, Vite, TanStack Query	АРМ, пульт преподавателя, админка, комната входа; очередь действий в IndexedDB с повтором после обрыва; общий расчёт времени shared/timing.ts
Обратный прокси	nginx (TLS, локальный CA)	HTTPS 8443, раздача SPA, /api и /ws к API
API	Python 3.12, FastAPI, SQLAlchemy async, asynccpg	REST по OpenAPI-контракту, WebSocket, RBAC и CSRF, аудит мутаций, конечный автомат карточки
Worker	тот же образ, python -m app.worker	планировщик выдачи карточек, очередь заданий (генерация, оценка, транскрипция), ИИ-судья, heartbeat
Оценщик	пакет evalcore (чистый Python)	критерии оценки, агрегирование, расчёт времени I-TIME, проверка эталона, генератор сценариев, парсер классификатора
База данных	PostgreSQL 16	занятия, карточки, журнал событий (append-only), оценки и переопределения, прогнозы, материалы, аудит
Локальная LLM	llama.cpp server + Qwen3-4B Q4_K_M (профиль 11m)	ИИ-оценка комментария, нарратив сценария; выключается AI_PROVIDER=off
Backup	контейнер на PostgreSQL 16	ежедневный pg_dump + архив пользовательских файлов, SHA-256, восстановление в отдельный проект

Взаимодействие:

- браузер ↔ nginx ↔ API по HTTPS (REST) и WSS (события `card.appeared/updated`, `evaluation.*`, `training.updated`, синхронизация часов `clock.ping/pong.v2`);
- API и worker пишут в PostgreSQL и публикуют NOTIFY; API держит LISTEN и рассылает события авторизованным клиентам; при потере LISTEN клиенты получают свежий snapshot;
- worker ↔ llama.cpp по OpenAI-совместимому HTTP внутри Docker-сети;
- все контракты стыков — в `contracts/` (OpenAPI, JSON Schema, WS-события, I-TIME и др.).

1.4 Данные

- **Классификатор происшествий 046_24** (1283 типа, 3835 правил оповещения шести служб) — справочник типов и служб-получателей; парсер определяет редакцию по шапке.
- **Сценарии**: карточка 112 + эталон (решение, статусы, службы, адрес, звонок, комментарий), уровень 1–4 и вес 1–10, происхождение (шаблон, ИИ, импорт по билетам, обучаемый). 10 сценариев построены по билетам заказчика (32 билета / 96 задач), эталоны — учебная разметка команды, до проверки преподавателем — черновики.
- **Журнал событий карточки** — неизменяемый, с временем клиента и сервера и ссылкой на проверенный образец часов; из него восстанавливаются replay, время и оценка.
- **Снимок занятия**: нормативы, веса и методика времени фиксируются при создании занятия; последующие изменения настроек прошлые занятия не меняют.

2. Методы обработки данных

Все формулы реализованы в коде и покрыты тестами; ссылки указывают на файлы репозитория.

2.1 Классификатор и маршрутизация

packages/evalcore/evalcore/classifier.py читает CSV классификатора (3 строки шапки), определяет редакцию по заголовку столбца 12 («Главная служба» — 046_24, «Сценарий реагирования» — 046_11) и проверяет, что каждая колонка служб принадлежит своей группе (МЧС, МВД, СМП, МОСГАЗ, Мослифт, ЖКХ). Правило оповещения = (тип, служба, условие, текст). Условие получает стабильный признак trigger (default, no_access, offense, victims, victims_off_site, gasification), потому что номера колонок в редакциях различаются. Пустая ячейка и «нет реагирования» правил не создают.

2.2 Сценарии и эталоны

- **По билетам заказчика** (scripts/build_ticket_scenarios.py): ситуация и адрес переносятся вручную со скана PDF (OCR — только для поиска), тип и службы — по 046_24, эталон — по памятке ДДС. В data/ticket_scenarios/manifest.json сохраняются билет, задача, страница, дословный текст, версия классификатора и основание эталона; teacher_review = not_reviewed. Телефоны заявителей заменены учебными. Тест проверяет, что эталонное прохождение получает полный балл по маршруту, статусам и адресу.
- **Генератор** (evalcore/scenario.py): конструктор (тип × адрес × пострадавшие × осложнения × уровень × вес × seed) → карточка и эталон детерминированно; один seed — один результат. Службы — основная служба типа и правила классификатора по trigger. ИИ-нарратив не меняет эталон и получателей.
- **Утверждение:** эталон проверяется перед утверждением (evalcore/validation.py) — допустимая цепочка статусов, решение, получатель перенаправления, адресат обязательного звонка.

2.3 Оценка по правилам

evalcore/scoring.py: итог = $\sum(w_i \cdot s_i) / \sum w_i$ по семи критериям и двум критериям комментария (если их вес есть в снимке занятия); при критической ошибке итог не выше critical_cap (по умолчанию 0,5). Веса и нормативы — из снимка занятия.

Критерий	Балл s_i	Критическая ошибка
reaction_time	1 при $t \leq N$, иначе $\max(0, 2 - t/N)$; t — реакция I-TIME v3 (направление → первое «Принята/Не принята»), $N = 30$ с	нет
handling_time	та же шкала для активной обработки v3 (первое открытие → завершение минус подтверждённое ожидание), $N = 180$ с	нет
status_flow	LCS(эталон, факт) / $\max(\text{длин})$ по последовательности статусов	нет
routing	1 при совпадении цепочки решений и адресата перенаправления, иначе 0	да
required_fields	доля заполненных обязательных полей	нет
address	доля компонентов ручного адреса диспетчера (город, улица, дом, корпус, квартира), совпавших с эталоном после нормализации (регистр, «ё», «г.», «ул.», «д.», «корп.», «кв.»); похожая другая улица не засчитывается; без ручного адреса — поиск в комментарии («ул.»/«улица» не различаются, номер дома — после «д.»/«дом» или сразу за улицей: «Дубинская улица, 1»)	да, если ошибка в улице или доме
call	лучшее среднее по звонкам: соединение, адресат, звонок до срока	по эталону
spelling	$1 - \text{ошибки} / \max(4, \text{слова} / 5)$; пустой обязательный комментарий — 0	нет
comment_keywords	доля найденных пунктов: адрес, суть происшествия, решение, пострадавшие, слова эталона	нет

Время в критериях считается тем же расчётом, что в разборе и на экранах (раздел 2.5), по методике из снимка занятия; для старых занятий — по их версии. Маршрутизация: исправленная «Не принята → Принята» в принятом сценарии не считается ошибкой маршрута.

Критерии комментария (28.09, Q&A Q10, FR-4.2) — evalcore/comment.py, проверка текста — apps/api/app/core/spelling.py; без ИИ и без сети. Вес по умолчанию: грамотность 0,5, ключевые сведения 1. В занятиях, созданных раньше, весов нет — критерии не считаются, итоги прежние.

- *Грамотность*. Слово верно, если оно есть в морфологическом словаре OpenCorpora (pymorphu3, все формы слова), в словаре 112 (spelling_112.txt: «Росгвардия», «спецтранспорт», «пр-т»...) или в названиях улиц (справочник и адрес сценария). Не проверяются слова короче трёх букв, сокращения из заглавных (ДТП, ГИБДД) и слова с заглавной не в начале предложения — имена и названия. Варианты исправления — слова словаря на расстоянии одной правки (пропуск, лишняя буква, перестановка, замена; дешевле пропуск и соседняя клавиша) и разбиение слитного написания. Прогон по всем текстам data/: из 2103 разных слов словарь не знал 86, большинство — настоящие опечатки исходных данных; ложные срабатывания внесены в словарь 112.
- *Ключевые сведения* — те же пункты, что у ИИ-судьи (раздел 2.4), но по словам в начальной форме: улица — без исправления опечаток («Дубнинская» ≠ «Дубининская»), номер дома — после «дом», «д.» или сразу за улицей; суть — половина значимых слов класса происхождения; решение — слова направления сил или отказа; пострадавшие — если они есть в карточке; свои слова преподаватель задаёт в студии (reference.comment_keywords, варианты через «/»). Опечатка в остальных словах пункт не отменяет — за неё снижается грамотность.
- *Подсказки в АРМ*. Галочка «Подсказки орфографии» в настройках занятия (по умолчанию выключена, в самостоятельной практике включена): под комментарием подчёркиваются ошибки и предлагаются исправления той же проверкой (POST /spelling/check). Обучаемому — только в своём занятии с включённой галочкой.

2.4 ИИ-оценка комментария

apps/api/app/worker/judging.py, провайдер providers/judge/local_llamacpp.py, модель Qwen3-4B-Instruct-2507 Q4_K_M через llama.cpp (CPU):

1. Из карточки и решения эталона формируются ключевые факты: адрес (улица, дом), суть происшествия, принятое решение или причина отказа, пострадавшие. Текст эталона модели не передаётся.
2. Модель получает факты и комментарий обучаемого **как данные** (поле JSON; инструкции внутри игнорируются), возвращает по JSON Schema раскладку фактов на найденные/отсутствующие, понятность 0–1 и пояснение на русском; temperature 0.
3. Ответ отклоняется (оценка остаётся по правилам, статус partial), если факт выдуман, пропущен или повторён, ответ вне схемы или истёк таймаут (120 с). Найденный факт без опоры в тексте (общая основа слова; для адреса — номер дома) считается отсутствующим.
4. Полнота = найденные / все факты; llm = (полнота + понятность) / 2.
5. **Итог = 0,8 · rules + 0,2 · llm**, при критической ошибке — не выше critical_cap.
6. Обратная связь: до трёх последних решений преподавателя по тому же сценарию, созданных до текущей оценки, передаются как примеры; их ID сохраняются. Старые оценки не переписываются, будущие результаты не используются.

Замер на CPU (4 потока): 15–27 с на комментарий; полный комментарий — 1,0/1,0; «Горит что-то, выехали» — 0,0/0,3; попытка инъекции не выполняется (docs/C06_JUDGE_REPORT.md).

2.5 Учебное время (I-TIME v3)

evalcore/timing.py и тот же алгоритм в браузере (apps/web/src/shared/timing.ts) проверяются 42 общими числовыми примерами (contracts/examples/timing-v2.json, timing-v3.json).

- **Направление** карточки в ДДС фиксирует сервер (момент выдачи worker).
- **Реакция** = направление → первый статус «Принята/Не принята»; норматив 30 с (памятка ДДС). Открытие карточки реакцию не закрывает.
- **Полная обработка** = первое открытие → завершение (работы завершены, отказ от работ, перенаправление, окончательная «Не принята»).
- **Ожидание** — объединение подтверждённых интервалов ожидания учебного сообщения бригады (от ответа на звонок до выдачи реплики) внутри обработки.
- **Активная обработка** = полная – ожидание; норматив 180 с сравнивается с активной.

- Пример: направление 0, открытие 20, «Принята» 40, завершение 190 → реакция 40 (превышение), полная 170, активная 170; при ожидании 60 с активная 110.
- Отсутствующее событие даёт null и аномалию, а не ноль; отрицательный интервал — invalid. Превышение отмечается только при достоверном измерении.

Синхронизация часов (метод midpoint): клиент шлёт ping с `client_sent`, сервер сохраняет момент получения и отвечает тем же `server_at`; клиент фиксирует `client_received` и подтверждает образец. Поправка = $\text{server_at} - (\text{client_sent} + \text{client_received})/2$, проверка формулы ± 1 мс, $\text{RTT} \leq 2$ с, возраст образца ≤ 60 с. Время действия = `client_ts` + поправка; если оно в будущем ($> +250$ мс) или задержка буфера > 30 с — берётся время сервера с отметкой «оценочно». Проверенное время даёт качество `verified`, иначе `estimated`.

2.6 Прогноз и аналитика

- **Прогноз до попытки** сохраняется при выдаче карточки: $\text{IRT-1PL } P = 1/(1+e^{-(\theta-b)})$, $b = (\text{вес} - 5,5)/2,25$, ожидаемое время = $N \cdot (1,35 - 0,7 \cdot P)$ (`worker/scheduler.py`, модель `worker-irt-fallback-v1`); факт дописывается один раз после завершения.
- **Рекомендация уровня** (`evalcore/adaptive.py`, `rules-v1`): по пяти последним завершённым попыткам — средний итоговый балл (с учётом решения преподавателя), критические ошибки и подтверждённое время. Балл $\geq 0,8$ без критических ошибок и превышений — повысить уровень и вес на шаг; балл $< 0,6$, две попытки с критическими ошибками или большинство превышений — понизить; иначе оставить. До трёх попыток — холодный старт. Основание выдаётся словами.
- **Разбор попытки** (`GET /cards/{id}/analysis`): статус попытки, время `v3`, слои оценки (обязательны `rules` и `llm`; невыполненные помечены с причиной), переопределение преподавателя, счётчики ошибок (`null` — не проверено, 0 — ошибок нет), прогноз и факт.
- **Аналитика занятия**: по каждому обучаемому — попытки, оценённые, средний итоговый балл, средние реакция и обработка **только по проверенным измерениям** с числом наблюдений.

3. Сборка, установка, эксплуатация

3.1 Требования

- Docker Engine (Ubuntu) или Docker Desktop с WSL2 (Windows 10/11), Docker Compose ≥ 2.24 .
- CPU x86-64, ОЗУ от 8 ГБ (с локальной моделью — от 8 ГБ свободной памяти рекомендуется больше), диск от 10 ГБ; GPU не требуется.
- Браузер Chromium или Firefox актуальной версии на рабочих местах.

3.2 Установка из репозитория

```
git clone https://github.com/DTYUI1/OneOneTwo-OnTheLine.git && cd OneOneTwo-OnTheLine
cp .env.example .env          # задать JWT_SECRET ( $\geq 32$  байт), DEMO_PASSWORD, API_MODE=database
docker compose build
docker compose up -d --wait db
docker compose run --rm --no-deps migrate          # Alembic; дождаться завершения
docker compose run --rm --no-deps api python -m app.seed # справочники, классификатор, сценарии
docker compose run --rm --no-deps api python -m app.seed.training # учебные бригады и доклады
docker compose up -d --wait api worker web
```

Вход: <https://localhost:8443/app/login> (комната — <https://localhost:8443/>). Учётные записи: admin, teacher, trainee01...trainee05, пароль — DEMO_PASSWORD. Сертификат выдаёт локальный CA: `docker compose cp certs:/certs/ca.crt ./ca.crt`, затем доверять его в ОС/браузере. Для доступа из локальной сети до первого запуска задайте `WEB_BIND=0.0.0.0` и `CERT_HOSTNAMES=localhost,127.0.0.1,<IP сервера>`.

Seed идемпотентен: повторный запуск не перезаписывает правки и синхронизирует классификатор. `app.seed.training` добавляет по учебному сценарию на службу (номера 70004210...70004215) с озвученными докладами первой бригады: обучаемый направляет бригаду, звонит на её прямой номер из панели бригад и слышит доклады о выезде, прибытии, работах и завершении. Первое поколение (70004200...70004205, без доклада о завершении) выведено в архив и не раздаётся.

3.3 Локальная ИИ-модель (опционально)

```
mkdir -p models # файл Qwen3-4B-Instruct-2507-Q4_K_M.gguf (Apache-2.0) из офлайн-комплекта
# AI_PROVIDER=local в .env
docker compose --profile llm up -d --wait llm worker
```

Модель читается только из локального файла (`LLM_MODEL_FILE`), без загрузки из интернета при запуске. Без модели (`AI_PROVIDER=off`) система работает, ИИ-слой помечается как невыполненный.

3.4 Установка без интернета

На подготовительной машине `deploy/offline.py create --output <каталог> --model <gguf> --model-source ... --model-license ...` собирает образы, модель, файлы запуска и манифест с SHA-256. На целевой машине `deploy/offline.sh load` (Linux) или `deploy/offline.ps1` (Windows) проверяет хеши и загружает образы; далее `docker compose up --pull never --no-build` по шагам раздела 3.2. Сборщик отказывает при неполных входных данных (справочник улиц, голоса).

3.5 Резервное копирование и восстановление

- Контейнер backup ежедневно в 02:00 UTC делает `pg_dump` и архив пользовательских файлов (записи докладов, учебные материалы) с SHA-256; вручную — `docker compose run --rm --no-deps backup backup-once`; из админки — «Копии и корзина» → «Сделать копию сейчас» (запрос в таблицу backups, контейнер забирает его в течение минуты через backup-roll; браузер команд на сервере не выполняет). Все копии и их результат видны там же. Папку `BACKUP_DIR` держите на отдельном диске.
- Удалённые администратором учётные записи обезличиваются только после успешной копии, начатой позже удаления (ТЗ: не удалять критичные данные без резервного копирования).
- Восстановление — только в новый проект и пустые тома: `sh deploy/restore.sh <новый-проект> backup-YYYYMMDDTHHMMSSZ`; восстановление в рабочий проект и занятые тома отклоняется. Проверено на Linux:

счётчики, ревизия и SHA-256 файлов совпадают (deploy/C08_REPORT.md).

3.6 Обновление, диагностика, остановка

- Обновление: `git pull`, `docker compose build`, `docker compose run --rm --no-deps migrate`, затем `docker compose up -d --wait api worker web`. Откат схемы с новыми данными запрещён миграциями — используйте `backup/restore`.
- Состояние: `/api/health` (БД, ревизия схемы, `heartbeat worker`, ИИ-провайдер), раздел «Состояние системы» в админке, `docker compose ps`, `docker compose logs <сервис>`.
- Остановка без потери данных: `docker compose stop`; не используйте `down -v`, если данные нужны.
- Журналы: действия пользователей и изменения данных пишутся в `audit_log` в БД (бессрочно, входят в ежедневный backup). Журналы контейнеров ротируются (json-file, 30 файлов по 20 МБ на сервис). Для хранения stdout-журналов ≥ 6 месяцев выгружайте их на хосте по расписанию, например раз в сутки из cron: `docker compose logs --no-color --since 24h > /var/log/arm112/$(date +%F).log`.

3.7 Проверки разработчика

```
uv sync --all-packages --frozen && pnpm install --frozen-lockfile
DATABASE_URL_TEST=postgresql+asyncpg://... uv run pytest -q
uv run ruff check . && uv run ruff format --check . && uv run mypy
pnpm lint && pnpm typecheck && pnpm test && pnpm build
pnpm exec playwright test -c qa/playwright.config.ts # на поднятом стенде, E2E_DATABASE=1
```

CI (GitHub Actions) выполняет те же проверки и браузерный набор на свежей БД и после перезапуска с сохранённой историей.

4. Условия применения и ограничения

4.1 Условия

- Решение рассчитано на учебный класс: локальный сервер и до 23 рабочих мест в одной сети, без доступа в интернет во время работы. Внешние API не вызываются; ИИ — только локальная модель, выключаемая настройкой AI_PROVIDER.
- Роль обучаемого — диспетчер ДДС, получающий готовую карточку от оператора 112 (ответы кейсодержателя от 16.09). Приём входящего звонка заявителя вне рамок.
- Алгоритм работы служб универсальный (решение 24.09); специализация исключений 103/104 из памяти не реализуется.
- Методика учебных трёх минут (активная обработка без подтверждённого ожидания) — решение команды, а не норматив памяти; 30 секунд до первичного статуса — из памяти ДДС.
- Эталонные ответы к билетам заказчик не предоставляет: эталоны — учебная разметка команды, утверждаются преподавателем в студии. Исходного аудио заявителей нет — ошибки ввода адреса оцениваются по текстовым данным карточки.

4.2 Ограничения текущей версии (на 26.09.2026)

Область	Ограничение	Где отражено
Проверка адреса	ручной адрес сравнивается с эталоном точно после нормализации сокращений; опечатки в названии улицы не прощаются, справочник улиц не подключён	раздел 2.3
Грамотность, семантика, сведения бригады	слои не подключены, в разборе помечены с причиной	—
Рекомендация уровня	правилковая (rules-v1) по попыткам занятия, без калибровки на исторических данных; прогноз — упрощённая IRT-модель	раздел 2.6
ИИ-судья	15–27 с на комментарий на CPU (отдельный замер i5); в классе 23 APM на i7 после #48 все 6 проверенных оценок готовы, последняя наблюдалась через 138,7 с после начала опроса. Это не средняя задержка на карточку. Модель 4В изредка ошибается в раскладке фактов, частично компенсировано проверкой опоры	docs/C06_JUDGE_REPORT.md, qa/bench
Сценарии по билетам	10 из 96 задач; лифтовых и коммунальных ситуаций в билетах нет	data/ticket_scenarios/
Голоса Piper	только учебное некоммерческое использование: ruslan — CC BY-NC-SA 4.0 (перед коммерческим применением профиль voice-4 заменяется на CC0-голос), irina — CC BY-SA 4.0 по данным RHVoice и дистрибутива, у первоисточника нет отдельного файла лицензии	scripts/voices/PROVENANCE.md
Офлайн-комплект	не собран целиком без справочника улиц и голосов; установка «с нуля без сети» не проверена	deploy/C08_REPORT.md
Нагрузка	цели выполнены на i7-11800H / 16 ГБ (Windows + Docker Desktop) и на i5-1155G7 / 7,5 ГБ (Linux): 23 APM, отчёт ≤ 1,36 с, 100 пользователей — p95 ≤ 1,1 с, PostgreSQL — тысячи записей/с; через API при 23 клиентах с последовательными запросами — 81–103 событий/с; одновременный вход всех 100 — p95 3,2–3,8 с; сервер 32 ГБ не проверялся	qa/bench/results/
Отказоустойчивость	автоматический перезапуск контейнеров и восстановление WebSocket проверены; горизонтальное масштабирование и отказ узла — нет	C-08
Журналы безопасности ≥ 6 мес.	аудит мутаций хранится в БД бессрочно и входит в backup; журналы контейнеров ротируются, для хранения stdout ≥ 6 мес. нужна выгрузка на хосте по расписанию (раздел 3.6)	C-08
Проверенные платформы	Linux (Arch, Ubuntu в CI), Windows 11 + Docker Desktop (первый ПК); Chromium и Firefox; Яндекс.Браузер не проверялся	—
149-ФЗ / 152-ФЗ	требования учтены в проектировании (локальное хранение, минимум ПДн, вымышленные данные в сидах, TLS, RBAC, аудит); юридическая сертификация не заявляется	—

4.3 Персональные данные

В учебных данных — только вымышленные ФИО и учебные номера +7 900 000-....; в билетах заказчика телефоны заменены. Записи докладов и материалы хранятся локально, в логи ПДн не пишутся.

5. Перечень библиотек

Собран скриптом `scripts/submission/build_libraries.py` из фактически установленных зависимостей (`uv.lock`, `pnpm-lock.yaml`). Лицензии — из метаданных пакетов. Своих пакетов (`arm112-api`, `evalcore`, `arm112-scripts`) в таблицах нет.

5.1 Python (API, worker, оценщик, скрипты)

Библиотека	Версия	Лицензия	Назначение
alembic	1.16.5	MIT	работа приложения
annotated-types	0.8.0	MIT	работа приложения
anyio	4.15.1	MIT	работа приложения
argon2-cffi	25.1.0	MIT	работа приложения
argon2-cffi-bindings	26.1.0	MIT	работа приложения
asynpg	0.30.0	Apache License, Version 2.0	работа приложения
attrs	26.1.0	MIT	работа приложения
certifi	2026.7.22	MPL-2.0	работа приложения
cffi	2.1.1	MIT-0	работа приложения
charset-normalizer	3.5.1	MIT	работа приложения
click	8.5.0	BSD-3-Clause	работа приложения
coverage	7.16.1	Apache-2.0	разработка и проверки
cryptography	46.0.1	Apache-2.0 OR BSD-3-Clause	работа приложения
DAWG2-Python	0.9.0	MIT	работа приложения
fastapi	0.116.1	MIT License	работа приложения
greenlet	3.5.6	MIT AND PSF-2.0	работа приложения
h11	0.16.0	MIT	работа приложения
httpcore	1.0.9	BSD-3-Clause	работа приложения
httptools	0.8.0	MIT	работа приложения
httpx	0.28.1	BSD-3-Clause	работа приложения
idna	3.20	BSD-3-Clause	работа приложения
iniconfig	2.3.0	MIT	разработка и проверки
jsonschema	4.25.1	MIT	работа приложения
jsonschema-path	0.3.4	Apache-2.0	разработка и проверки
jsonschema-specifications	2025.9.1	MIT	работа приложения
lazy-object-proxy	1.12.0	BSD-2-Clause	разработка и проверки
Mako	1.4.1	MIT	работа приложения
MarkupSafe	3.0.3	BSD-3-Clause	работа приложения
mypy	1.18.1	MIT	разработка и проверки
mypy_extensions	1.1.0	MIT	работа приложения
openapi-schema-validator	0.6.3	BSD-3-Clause	разработка и проверки
openapi-spec-validator	0.7.2	Apache-2.0	разработка и проверки
packaging	26.3	Apache-2.0 OR BSD-2-Clause	разработка и проверки
pathable	0.4.4	Apache-2.0	разработка и проверки
pathspec	1.1.1	Mozilla Public License 2.0 (MPL 2.0)	разработка и проверки
pluggy	1.6.0	MIT	разработка и проверки

Библиотека	Версия	Лицензия	Назначение
pycparser	3.0	BSD-3-Clause	работа приложения
pydantic	2.13.5	MIT	работа приложения
pydantic-settings	2.10.1	MIT	работа приложения
pydantic_core	2.46.5	MIT	работа приложения
Pygments	2.21.0	BSD-2-Clause	разработка и проверки
PyJWT	2.10.1	MIT	работа приложения
pymorphy3	2.0.6	MIT license	работа приложения
pymorphy3-dicts-ru	2.4.417150.4580142	MIT license	работа приложения
pytest	8.4.2	MIT	разработка и проверки
pytest-cov	6.3.0	MIT	разработка и проверки
python-dotenv	1.2.3	BSD-3-Clause	работа приложения
python-multipart	0.0.20	Apache-2.0	работа приложения
PyYAML	6.0.2	MIT	работа приложения
referencing	0.36.2	MIT	работа приложения
requests	2.34.2	Apache-2.0	работа приложения
rfc3339-validator	0.1.4	MIT license	разработка и проверки
rpds-py	2026.6.3	MIT	работа приложения
ruff	0.13.0	MIT License	разработка и проверки
setuptools	84.0.0	MIT	работа приложения
six	1.17.0	MIT	работа приложения
SQLAlchemy	2.0.43	MIT	работа приложения
starlette	0.47.3	BSD-3-Clause	работа приложения
types-jsonschema	4.25.1.20250822	Apache-2.0	разработка и проверки
types-PyYAML	6.0.12.20250915	Apache-2.0	разработка и проверки
typing-inspection	0.4.4	MIT	работа приложения
typing_extensions	4.16.0	PSF-2.0	работа приложения
urllib3	2.8.0	MIT	работа приложения
uvicorn	0.35.0	BSD-3-Clause	работа приложения
uvloop	0.22.1	MIT License	работа приложения
watchfiles	1.2.0	MIT	работа приложения
websockets	17.1	BSD-3-Clause	работа приложения

5.2 JavaScript — в сборке веб-приложения

Библиотека	Версия	Лицензия
@remix-run/router	1.23.0	MIT
@tanstack/query-core	5.87.1	MIT
@tanstack/react-query	5.87.1	MIT
ajv	8.17.1	MIT
ajv-formats	3.0.1	MIT
fast-deep-equal	3.1.3	MIT
fast-uri	3.1.8	BSD-3-Clause
idb	8.0.3	ISC
js-tokens	4.0.0	MIT

Библиотека	Версия	Лицензия
json-schema-traverse	1.0.0	MIT
loose-envify	1.4.0	MIT
openapi-fetch	0.14.0	MIT
openapi-typescript-helpers	0.0.15	MIT
react	18.3.1	MIT
react-dom	18.3.1	MIT
react-router	6.30.1	MIT
react-router-dom	6.30.1	MIT
require-from-string	2.0.2	MIT
scheduler	0.23.2	MIT

5.3 JavaScript — инструменты разработки (прямые зависимости)

Библиотека	Версия	Лицензия
@eslint/js	9.35.0	MIT
@types/node	22.18.1	MIT
@types/react	18.3.24	MIT
@types/react-dom	18.3.7	MIT
@vitejs/plugin-react	4.7.0	MIT
eslint	9.35.0	MIT
fake-indexeddb	6.1.0	Apache-2.0
openapi-typescript	7.9.1	MIT
prettier	3.6.2	MIT
typescript	5.8.3	Apache-2.0
typescript-eslint	8.42.0	MIT
vite	6.3.6	MIT
vitest	3.2.4	MIT

Всего JS-пакетов с транзитивными: 385; по лицензиям: MIT — 310, Apache-2.0 — 36, ISC — 22, BSD-2-Clause — 8, BSD-3-Clause — 4, Python-2.0 — 1, CC-BY-4.0 — 1, CC0-1.0 — 1, 0BSD — 1, (MIT OR CC0-1.0) — 1.

5.4 Модели, данные и образы

Компонент	Версия/источник	Лицензия	Как поставляется
Qwen3-4B-Instruct-2507, GGUF Q4_K_M	unsloth/Qwen3-4B-Instruct-2507-GGUF	Apache-2.0	файлом models/...gguf, не в Git; deploy/offline.py
llama.cpp server	ghcr.io/ggml-org/llama.cpp:server@sha256:9dc0a0f4...	MIT	образ arm112-11m:0.1.0
Морфологический словарь русского языка (проверка грамотности и ключевых слов, 28.09)	OpenCorpora через pymorphy3-dicts-ru 2.4.417150	пакет — MIT, данные OpenCorpora — CC BY-SA	в образе API (пакет Python), без сети; словарь 112 — apps/api/app/core/spelling_112.txt
PostgreSQL	16.10 (образ Compose)	PostgreSQL License	Docker-образ
nginx	образ Compose web	BSD-2-Clause	Docker-образ
Piper TTS, голоса denis/dmitri/irina/ruslan	PR #16 (B-01)	CC0 / CC0 / не указана / CC BY-NC-SA 4.0	решение капитана по irina/ruslan открыто

Компонент	Версия/источник	Лицензия	Как поставляется
Классификатор 046_24, билеты, памятка ДДС	материалы кейсодержателя	права не установлены	docs/dataset/, только для хакатона

6. Матрица «требование → реализация → проверка»

Статусы на 26.09.2026 по main: **есть** — реализовано и покрыто проверками; **частично** — реализована часть, остаток указан; **нет** — не реализовано. Замеры нагрузки — qa/bench: i7-11800H / 16 ГБ (Windows + Docker Desktop) и i5-1155G7 / 7,5 ГБ (Linux). Строки FR-4.4, FR-7.1...7.4 и NFR-12 обновлены 27.09.2026 по ветке dev (PR #55, C-07); FR-7.1 и FR-7.2 — 28.09.2026 (корзина и копия по кнопке, ветка feature/dwots-trash); FR-4.2 и FR-4.4 — 28.09.2026 (грамотность и ключевые сведения комментария).

Функциональные требования

ID	Требование	Статус	Реализация	Проверка
FR-1.1	Строка состояния, новые карточки в реальном времени	есть	apps/web/src/arm, WS card.appeared, worker scheduler.py	e2e qa/e2e, test_c03_sessions.py
FR-1.2	Карточка «формата 112»	есть	IncidentCard.tsx, contracts/card.schema.json	e2e, cardModel.test.ts
FR-1.3	Принять/не принять/перенаправить, статусы с комментариями	есть	APM: этапы «Прибытие» и «Проведение работ», возврат «Не принята → Принята», причина отказа до отправки (D-03); сервер — универсальный автомат v3, 422 reason_required	test_c01_v3_statuses.py, vitest APM, e2e
FR-1.4	Таймеры 30 с / 3 мин	есть	живые таймеры APM, live-доски и replay через общий shared/timing.ts по методике занятия (v3: реакция до первого решения, норматив по активной обработке)	42 примера в Python и TS, vitest board, timeline
FR-1.5	Параллельные карточки	есть	лимит parallel_cards, очередь выдачи	test_c03_sessions.py
FR-1.6	Подсказки нулевого уровня	есть	«?» у статуса, номера наряда, комментария и адреса при hints_level ≥ 1, событие hint_open (без эталона)	vitest APM, e2e
FR-1.7	Справочная база обучаемому	есть	«Справка» в APM: назначенные занятию материалы точной версии; загрузка и назначение — у преподавателя	test_c07_materials.py, vitest materialsModel
FR-1.8	Ручной выбор бригад	есть	панель бригад софтфона на API C-04: бригады своей ДДС, прямые номера, call_dial_target	logic.test.ts, test_c04_training.py, e2e brigade-audio
FR-2.1	Софтфон, набор 3-значного номера	есть	apps/web/src/arm/softphone	vitest, e2e
FR-2.2	Реплики адресата TTS, муж./жен. голоса	есть	предзаписанные реплики Piper, 4 голоса (3 муж., 1 жен.); лицензии и ограничения — scripts/voices/PROVENANCE.md	voices.test.ts, прослушивание 24.09
FR-2.3	Факт, время, адресат звонка; запись WAV/WebM	есть	журнал звонков, POST /calls/{id}/audio	test_lifecycle.py
FR-2.5	Сообщения бригады и журнал предъявления	есть	worker выдаёт доклады по плану, аудио через API, предъявление по концу звука или «Прочитал»	test_c04_*.py, evidence.test.ts, e2e brigade-audio
FR-3.1	Шаблонный конструктор	есть	студия D-01 на API сценариев; API предпросмотра previewScenario	vitest студии, test_c05_content_api.py
FR-3.2	ИИ-генерация с эталоном	частично	пакеты генерации (шаблон/ИИ-нарратив), эталон детерминирован	test_evaluations.py, test_llm_*
FR-3.3	Предпросмотр, комментарий, частичное утверждение	частично	предпросмотр и утверждение пакета в студии (API previewScenario, approvePack); правка по комментарию через ИИ (reviseScenario) — нет	test_c05_content_api.py, vitest студии

ID	Требование	Статус	Реализация	Проверка
FR-3.4	Уровень 1–4, вес 1–10	есть	поля сценария, suggested_weight генератора	схемы, тесты генератора
FR-3.5	Предгенерация пакета	есть	POST /packs/generate, worker, прогресс	test_c05_content_api.py
FR-3.6	Источники: 96 задач, памятка, 046_24	частично	046_24 подключён, 10 сценариев по билетам с происхождением	test_ticket_scenarios.py, test_classifier.py
FR-3.7	Категории: система/обучаемые/смесь	частично	origin сценария; выбор смеси в пульте не реализован	—
FR-4.1	Детерминированные критерии	есть	evalcore 7 критериев, веса, critical_cap; время — по I-TIME v3 из снимка занятия	packages/evalcore/tests (162), test_evaluations.py
FR-4.2	Опечатки, адрес относительно основания	есть	ручной адрес сравнивается с эталоном после нормализации сокращений, регистра и «ё»; похожая другая улица не засчитывается; грамотность комментария — критерий spelling в момент оценки (словарь OpenCorpora + словарь 112 + улицы, без ИИ) с вариантами исправления в разборе; подсказки в АРМ — галочкой преподавателя	test_scoring.py (V-02), test_comment.py, test_spelling.py, test_c06_analysis.py
FR-4.3	ИИ-оценка свободного текста с объяснением	есть	локальный судья Qwen3-4B, факты + понятность	test_c06_judge.py, реальный замер
FR-4.4	Настраиваемые веса, итоговый балл	есть	нормативы и веса класса — администратор; для своего занятия — преподаватель («Новое занятие», снимок settings_snapshot); формула rules + ИИ; критерии комментария (грамотность, ключевые сведения) со своими весами — и без ИИ	test_database.py, test_c06_judge.py, test_comment.py, vitest adminModel
FR-4.5	Объяснимость, «согласен/не согласен», приоритет преподавателя	есть	overrides, effective_total, журнал	test_evaluations.py, test_c06_analysis.py
FR-4.6	Фидбэк и материалы в следующем цикле	частично	few-shot из overrides; загрузка материалов; использование evaluation-материалов — нет	test_c06_judge.py, test_c07_materials.py
FR-4.7	Батчевая оценка после сессии	есть	worker-задание evaluate	test_evaluations.py
FR-4.8	Оценка по предъявленным сведениям	частично	журнал предъявления есть, критерий — V-02	test_c04_training.py
FR-4.9	Не штрафовать за недоступное обучаемому	частично	эталоны по билетам строятся по видимым данным; критерий — V-02	манифест сценариев
FR-5.1	Пульт, раздача, старт/стоп	есть	пульт D-01, адресная пачка C-03, finish v2	e2e team-integration, test_c03_*
FR-5.2	Live-мониторинг	есть	live-доска, replay	vitest board, timeline, e2e
FR-5.3	Адаптивность	есть	прогноз до попытки (IRT-IPL); объяснимая рекомендация уровня «повысить / оставить / понизить» по последним попыткам, холодный старт	test_adaptive.py, test_c06_analysis.py
FR-5.4	Обучаемому только его события	есть	RBAC и фильтрация по участнику	test_c03_access.py, e2e session-isolation
FR-5.5	Загрузка материалов, проверка грамматики	частично	загрузка PDF/DOCX/JSON/XML/CSV с проверкой, версии, назначение занятию — есть; проверка грамматики — V-02	test_c07_materials.py, vitest materialsModel
FR-6.1	Отчёт по обучаемому	есть	отчёт D-02, личные результаты, GET /cards/{id}/analysis	test_evaluations.py, test_c06_analysis.py
FR-6.2	По группе: лидеры/отстающие и почему	есть	отчёт преподавателя по аналитике занятия, графики, причины отличий и рекомендация уровня с	vitest reportModel, test_c06_analysis.py

ID	Требование	Статус	Реализация	Проверка
			основанием по каждому обучаемому	
FR-6.3	Экспорт CSV	есть	GET /reports/session/{id}/csv (UTF-8 BOM, ;), пустая ячейка вместо 0 для неоценённого	test_evaluations.py, test_report_unevaluated.py
FR-6.4	Отчёт ≤ 30 с	есть	JSON / CSV / аналитика на 69 карточках 23 обучаемых: i7 — 0,04 / 0,03 / 0,64 с (с ИИ — до 1,36 с), i5 — 0,03 / 0,02 / 0,62 с	qa/bench/results/
FR-7.1	Учётные записи, RBAC, блокировка, аудит	есть	учётки всех ролей: создание, правка, роль, пароль, блокировка с отзывом сессий и закрытием WebSocket; удаление — в корзину, обезличивание только после резервной копии (ТЗ: не удалять критичные данные без копии), до неё — «Восстановить»; RBAC x-roles; администратор вне учебного процесса (31 операция, WS без учебных событий); журнал аудита с фильтрами, неудачные входы	test_c07_users.py, test_c07_trash.py, test_c07_admin_limits.py, test_c07_audit.py, test_database.py
FR-7.2	Бэкап ежедневно, процедуры	есть	контейнер backup, restore.sh; копия по кнопке администратора («Копии и корзина»: запрос в таблицу backups, backup-poll раз в минуту) и история всех копий с результатом; последняя копия, её свежесть и число копий — в «Состоянии системы», предупреждение, если копий нет или они старше суток	deploy/C08_REPORT.md (Linux, Windows), test_c07_status.py, test_c07_trash.py, ручной прогон backup-poll/backup-once/restore-into-empty на PostgreSQL 16 (28.09)
FR-7.3	Журналы ≥ 6 мес., TLS	частично	TLS с локальным СА; аудит действий в БД бессрочно, только дописывается (триггер), в ежедневном backup и в разделе «Журнал аудита» с датой самой ранней записи; ротация журналов контейнеров (30 × 20 МБ); хранение stdout ≥ 6 мес. — выгрузка на хосте по расписанию (раздел 3.6)	docker compose config, test_c07_audit.py
FR-7.4	Мониторинг и оповещение	есть	«Состояние системы»: база, обработчик, канал обновлений, очередь, нагрузка, память, диск, копии; оповещения собирает сервер: плашка сбоя в шапке администратора и преподавателя, предупреждения на всех страницах администрирования. Внешних каналов (почта) нет — контур без интернета	test_database.py, test_c07_status.py, vitest adminModel

Нефункциональные требования

ID	Статус	Основание
NFR-1 Локальность	есть	внешних вызовов нет; LLM из локального файла, pull_policy: never
NFR-2/3 Железо, CPU-only	частично	CPU-only, GPU не нужен; замеры на i7-11800H / 16 ГБ и i5-1155G7 / 7,5 ГБ — цели выполнены; сервер 32 ГБ из NFR-2 не проверялся
NFR-4 Нагрузка	есть	23 APM одновременно, 69 карточек без ошибок; 100 пользователей онлайн и 20 активных карточек — p95 рабочих запросов 1,07 с (i7) / 0,85 с (i5), цель ≤ 2 с; одновременный вход всех 100 — p95 3,2–3,8 с
NFR-5 Отклик ИИ	есть	асинхронно: 15–27 с на комментарий (отдельный замер i5); в классе 23 APM на i7 после #48 — 6/6 оценок готовы, последняя наблюдалась через 138,7 с после начала опроса (до #48 — 264,6 с); p95 выдачи карточки с ИИ 3,87 с против 23,32 с ранее. Средняя задержка одной оценки в этом прогоне не измерялась
NFR-6 Голос	частично	симуляция на предзаписанных репликах Piper, задержка VoIP не заявляется
NFR-7 Устойчивость	частично	очередь действий в IndexedDB и повтор, восстановление WS и LISTEN, автоперезапуск; отказ узла не проверен
NFR-8 БД PostgreSQL	есть	PostgreSQL 16: 3452–8277 записей/с (pgbench, строка в отдельной транзакции, 8 соединений); через API при 23 клиентах, каждый по событию за раз — 81–103 событий/с (задержка запроса, не БД). См. пояснение к старым отчётам в qa/bench/results/README.md
NFR-9 ОС/браузеры	частично	Linux, Windows + Docker Desktop, Chromium и Firefox; Яндекс.Браузер не проверялся

ID	Статус	Основание
NFR-10 Форматы	есть	JSON-сценарии со схемами, CSV-отчёты, WAV/WebM записи, PDF/DOCX/XML/CSV материалы с проверкой содержимого, SQL-дамп
NFR-11 Простота	есть	прозрачные правила + одна лёгкая локальная модель
NFR-12 Масштабируемость	частично	службы и типы из данных; администратор включает и выключает службы без правки кода (C-07); горизонтальное масштабирование не проверено
NFR-13 Русский интерфейс	есть	все тексты UI и ошибок на русском
NFR-14 Нормативка, открытый код	частично	открытый код без обфускации, ПДн минимизированы; сертификация не заявляется

7. Дополнительный модуль «Оператор 112»

Модуль — дополнение сверх обязательного скоупа задачи № 9. По ответам кейсодержателя от 16.09 обучаемый — диспетчер ДДС, а приём звонка заявителя оператором 112 в рамки задачи не входит (раздел 4.1). Модуль показывает, как тот же тренажёр учит первое звено цепочки — оператора, который принимает вызов, расспрашивает заявителя и заполняет карточку. Основной тренажёр ДДС модуль не меняет: у него свой маршрут /operator, свой API /api/operator/* (вне contracts/openapi.draft.yaml), попытки хранятся в общем журнале аудита.

Материалы: контракты диалога, опросной карты, оценки и голоса — contracts/operator/, сценарии и логика — apps/api/app/operator/, экран — apps/web/src/operator/.

7.1 Путь обучаемого

1. Вход — плитка «112» на экране «Выберите обучение» (обучаемый; ссылка «Оператор 112» в шапке открывает этот же экран, крестик на нём — выход к экрану входа) или ссылка «Оператор 112» в шапке (преподаватель).
2. «Входящий вызов 112»: номер АОН, выбор звонка — случайный или конкретный персонаж, переключатель проводника первого звонка, кнопка «Принять вызов» (она же разрешает браузеру звук).
3. Экран карточки — по снимкам АРМ Системы 112 (docs/screenshots/card_112): телефоны и таймер, заявитель, адрес, описание, тип происшествия и опросная карта, полоса служб. Учебные дополнения — отдельным слоем: окно разговора, проводник, справка «?». Окно открывается по центру: его можно переносить за заголовок и менять размер за любой край или угол (подсказка — в нижней полосе окна). Кнопки переноса и размера работают стрелками. Положение и размер запоминаются в браузере, а каждый звонок начинается с развёрнутого разговора; «В центр» возвращает окно в середину экрана. Карточка занимает всю высоту колонки. Чтобы открыть поля под разговором, перенесите или сверните его — проводник тогда подскажет, как развернуть разговор обратно. У разбора отдельная шапка: крестик закрытия расположен над прокруткой.
4. Разговор: заявитель говорит голосом, паникует, успокаивается от приёма «Успокоить», переспрашивает на паузе, злится на лишние вопросы; посреди звонка ситуация ухудшается.
5. Вопросы — по шагам стандарта приёма вызова: адрес → номер для обратной связи → что случилось → угроза людям → детали и доступ. На шаге 3–4 правдоподобных варианта; режим «Сложно» — свой вопрос словами.
6. «сохранить» — оценка по восьми критериям с пояснениями и разбор звонка: лента разговора с пометками «помогло / навредило / внимание», время шагов, эталонный порядок, «Пройти заново».
7. Выход — крестик на полосе служб ведёт на экран «Результаты тренировок «Оператор 112»»: сводка (попыток, средний и лучший балл, по каждому звонку), список попыток и разбор выбранной, «Новый звонок». Крестик экрана результатов возвращает обучаемого к выбору обучения («112» или «Диспетчер служб»), преподавателя — в его кабинет.

7.2 Персонажи

Персонаж	Сценарий	Что отрабатывает
Девушка, 22 года	пожар: дым из квартиры соседки	истерика — вопросы не слышны без «Успокоить»; ловушка на слух «Дубнинская / Дубининская» (Q&A 16.09) ловится повтором адреса
Мужчина, 45 лет	ДТП с пострадавшими, разлив топлива	деловой темп, адрес на проспекте — дом и ориентир; дым из-под капота — совет отойти
Пожилая женщина, 72 года	мужу плохо с сердцем	медленная речь, переспрашивает; муж перестаёт отвечать — совет проверить дыхание
Мальчик, 9 лет	застрял в лифте без взрослых	не знает номера дома — ориентир и табличка на доме; погас свет — удержать на связи; службы 101 и Мослифт

Ещё три сценария (газ, человек в опасности, ошибочный звонок) — вне ротации, data/operator/archive/.

7.3 Методы

- **Заявитель по правилам**, без генерации и без сети (`apps/api/app/operator/caller.py`, `contracts/operator/dialog.md`). Паника 0–3: с 2 ответы панические и факт не засчитан, 3 — истерика. Выше порога истерии помогает только просьба с причиной, повторённая подряд (метод повторяющейся настойчивости IAED). Вопрос раньше адреса, лишний вопрос, повтор и повторный переспрос после пауз по 18 с поднимают панику. Первый переспрос после действия оператора не повышает панику; каждое действие обнуляет серию переспросов. Одинаковые действия дают одинаковый ответ.
- **Шаги опроса** (ARCO/NENA): у каждого вопроса опросной карты — шаг 1–5; шаг пройден, когда заявитель услышал все его вопросы. Лишние вопросы — правдоподобные для своего шага (почтовый индекс, домашний телефон, марка машины), у каждого — ответ в манере персонажа.
- **Повтор адреса вслух**: оператор повторяет адрес из карточки, заявитель сверяет его с тем, что знает сам, — улицу и те дом, корпус, квартиру, что ему известны; сравнение — как в оценке адреса (`evalcore.address`).
- **Свой вопрос** («Сложно») сопоставляется с опросной картой по синонимам с грубой нормализацией окончаний (`match_question`).
- **Оценка попытки** (`apps/api/app/operator/evaluation.py`, `contracts/operator/evaluation.md`) — детерминированная, сумма весов 100:

Критерий	Вес	Что проверяет
Тип происшествия	15	совпадение с эталоном сценария
Теги	10	отметки, от которых зависят службы
Службы	15	полоса служб против <code>routing_rules</code> классификатора; лишние — штраф
Адрес	20	поля адреса; неверная улица — весь критерий 0; округ и район — только если заявитель их называл
Описание	15	доля явных фактов сценария (5–7, с синонимами) в описании своими словами
Порядок опроса	10	адрес первым (0,4), подтверждён повтором (0,2), номер для обратной связи (0,2), ориентир (0,2)
Работа с заявителем	10	итоговая паника, успокоение просьбой с причиной, паузы, совет при ухудшении
Время до сохранения	5	норматив 180 с — только при верных типе и адресе

- **Журнал звонка**: каждое действие оператора и ответ заявителя с пометкой, паникой до и после и временем; сервер проверяет журнал и хранит его в попытке — разбор открывается и из списка попыток.

7.4 Без интернета и GPU

Голоса заявителей озвучены заранее Piper (`piper-tts 1.2.0`, голоса `irina` и `ruslan`) в контейнере: 186 реплик в спокойном, паническом и истерическом стиле, телефонная полоса 300–3400 Гц, Ogg/Opus; фон в трубке (подъезд с датчиком дыма, улица, квартира, лифтовая шахта) — синтез `ffmpeg`, без сторонних записей; всего около 1,6 МБ (`data/voices/operator/`, происхождение — `scripts/voices/PROVENANCE.md`). Нет файла или звук выключен — реплика видна текстом, темп разговора держится паузой по длине текста.

7.5 Проверки

`pytest` модуля (`apps/api/tests/test_operator_*.py`), `vitest` экрана (`apps/web/src/operator/*.test.ts`), `e2e` на стенде с БД — `qa/e2e/operator-*.spec.ts` в Chromium и Firefox: вход, живой звонок, шаги и варианты, «Сложно», повтор адреса, разбор, «?», проводник (новичок проходит звонок девушки только по его подсказкам), выбор звонка, раскладка на окнах 1366×768, 1536×864, 1920×1080 и на высотах с панелью браузера 1366×650, 1536×730.

7.6 Ограничения

- Четыре сценария, по одному на персонажа; редактора сценариев модуля нет.
- Своих таблиц модуля нет (история OP-01): сценарии — в общей таблице настроек, попытки — в журнале аудита; преподаватель попытки модуля в пульте не видит.
- Карточка оператора сохраняется в формате `contracts/card.schema.json`, но в АРМ ДДС не уходит: цепочка «112 → ДДС» (OP-09) — после хакатона.
- Свой вопрос узнаётся по синонимам, без языковой модели: необычная формулировка может быть не понята — заявитель переспросит.
- Голоса ребёнка и пожилой женщины — один синтетический голос со сдвигом тона и темпа.
- Вид экрана один в один с АРМ 112 при наложении на скриншоты — следующий этап доработки.